

Reviewer Pack — Minimal Order of Brauer Groups Bounds Read-Twice BP Size

Entry 949e268cc975 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-29 07:27:35 UTC

Minimal Order of Brauer Groups Bounds Read-Twice BP Size

Entry ID: 949e268cc975 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-29 07:27:35 UTC

1. Conjecture

Field A (mathematical branch): Algebra (Brauer Groups) **Field B** (complexity object): Boolean Function Complexity (Read-Twice Branching Program Complexity)

Statement:

[‘For any Boolean function f with n variables, the minimal order of its Brauer group over the finite field $\text{GF}(p)$, denoted as $|\text{Br}(G_f)|$, is upper bounded by the read-twice BP size of f , i.e., $|\text{Br}(G_f)| \leq \text{RTBP}(f)$.’, ‘For all instances of boolean functions with $n \leq 40$ variables and any prime p , if $|\text{Br}(G_f)| > \text{RTBP}(f)$, then there exists a counterexample with RTBP size less than or equal to $|\text{Br}(G_f)|$.’, ‘The minimal order of Brauer groups for boolean functions is computable in polynomial time using the Brauer-Weil theory.’]

Rationale (proposer’s reasoning):

[‘Brauer groups provide an algebraic invariant that captures non-trivial linear behavior in finite fields. This could potentially reveal deep structural properties of Boolean functions that are not evident from their circuit complexity.’, ‘The minimal order of Brauer groups is computable, which makes it feasible to test the conjecture on small instances within the given time constraints.’, ‘Read-twice BP size is a well-established complexity measure for boolean functions, and bounding it using algebraic invariants like Brauer groups could provide new insights into complexity theory.’]

Taxonomy category: BrauerGroupsToReadTwiceBP (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): a585626893ac4cae

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, across all $n \leq 40$ boolean functions and primes p , $|\text{Br}(G_f)|/\text{RTBP}(f)$ ratio is less than or equal to 1 for at least 80% of seeds. It is falsified if any seed results in a ratio greater than 1 for more than 20% of the functions.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - 'Brauer Groups' AND 'Read-Twice BP size' - 'Minimal order' AND 'Brauer group' AND 'Boolean function complexity' - 'Polynomial time computable' AND 'Brauer-Weil theory' AND 'read-twice BP size'

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=240.4s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_boolean_function(n):
        return [random.choice([0, 1]) for _ in range(2**n)]

    def read_twice_bp_size(f):
        n = len(f)
        if n <= 1:
            return 0
        max_value = -math.inf
        for i in range(1 <= (n-1)):
            value = f[i] + f[~i]
            if value > max_value:
                max_value = value
        return max_value

    def minimal_order_brauer_group(f):
        n = len(f)
        # Simplified Brauer group order calculation for demonstration purposes
        return 2**n

    instances_tested = 0
    total_ratio = 0.0
    conjecture_holds = True
    counterexample = ""
```

```

for _ in range(30):
    n = random.randint(5, 40)
    f = generate_boolean_function(n)
    rtbp_size = read_twice_bp_size(f)
    brauer_group_order = minimal_order_brauer_group(f)

    if brauer_group_order > rtbp_size:
        conjecture_holds = False
        counterexample = f"n={n}, RTBP size={rtbp_size}, Brauer group order={brauer_group_order}"
        break

    instances_tested += 1
    total_ratio += brauer_group_order / rtbp_size

if not conjecture_holds:
    return {
        "metric_name": "ratio",
        "metric_value": total_ratio / instances_tested,
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

return {
    "metric_name": "ratio",
    "metric_value": total_ratio / instances_tested,
    "instances_tested": instances_tested,
    "conjecture_holds": conjecture_holds,
    "counterexample": ""
}

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] if len(sys.argv) > 1 else [2**i + 3 for i in range(5, 8)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_ratio = sum(r["metric_value"] for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_ratio} std=0.0 support_fraction=1.0")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_ratio} std=0.0 support_fraction={support_fraction:.2f}")
    else:
        first_failing_seed = next(i for i, r in enumerate(results) if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"{results[first_failing_seed]['counterexample']}\"")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, which means we cannot verify the conjecture's support or falsification based on the pre-registered criteria. | next: Run the test again with increased time limits to ensure it completes and produces results.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12523
2	preregistration	ollama_remote	glm4:latest	0	0	5836
3	novelty	ollama_remote	glm4:latest	0	0	4899
4	novelty	ollama_remote	glm4:latest	0	0	5312
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	40580
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	6798
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12325
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8701
9	judge	ollama_remote	glm4:latest	0	0	25836

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 122810 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in *research/audit/949e268cc975.jsonl*)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in *research/audit/949e268cc975.jsonl* for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: *research/pvsnp_sandbox/test_*.py* (per-cycle)

Replay tarball: *research/replay/949e268cc975.tar.gz* (if generated)